DEV_IMPL_GUIDE - DD: OSR-RMA-01-FLOW-EQU — Equipment Upgrade

1. Purpose

This guide explains how to implement `DD_OSR-RMA-01-FLOW-EQU-v1.0.md` as executable SOPHIX behavior.

Use it with:

- `DD_OSR-RMA-01-FLOW-EQU-v1.0.md`
- `DD_SUB-WF-FRAMEWORK-01-Subscription_Workflow_Framework-v1.0.md` when this DD participates in subscription workflow operations.
- `DD_SUB-WF-FRAMEWORK-01-WORKERS-v1.0.md` when this DD references worker topics.

2. Runtime Actors

Runtime component	Responsibility
API pod / module service	Receives requests, validates, loads authoritative state, starts commands.
Owning module database	Stores tables/configuration owned by this DD.
Reference modules	Provide data through approved APIs/client wrappers.
Camunda	Runs BPMN process when this DD is workflow-driven.
Worker apps	Execute logical worker topics.
Drools	Evaluates packages when rules are declared.
Kafka/outbox	Publishes success/failure and integration events.

3. Before Implementation

Confirm these contracts from the DD:

Contract	Value
Endpoint scan reference	`GET /api/swap-requests/swap_01HZ_MUTHONI_UPG`
BPMN/process key	`configured BPMN process`
Drools package	`configured Drools package`
Tables	No CREATE TABLE block declared
Events	No explicit events declared

4. Execution Flow

1. Caller sends the request to the public or internal endpoint.
2. API pod validates payload shape, authentication, actor role, and idempotency.
3. API pod loads authoritative state from the owning module and approved reference APIs.
4. API pod loads operator/module configuration and catalog rows.
5. API pod maps facts and calls Drools where the DD requires rules.
6. API pod calls the shared framework start contract with an internal command.
7. Framework creates the operation row, resolves the process key, and starts Camunda.
8. BPMN executes service tasks by worker topic.
9. Message correlators wake the BPMN for async payment, fulfillment, or work-order events.
10. BPMN commits authoritative state.
11. BPMN writes events through outbox and finalizes the operation.

5. API And Command Handling

Endpoints found in the DD:

- `GET /api/swap-requests/swap_01HZ_MUTHONI_UPG`
- `POST /api/billable-events`
- `POST /api/equipment-instances/inst_01HZ_CM_MUTHONI_DOCSIS30/lifecycle-events`
- `POST /api/equipment-instances/inst_01HZ_CM_MUTHONI_DOCSIS31/lifecycle-events`
- `POST /api/swap-requests/equ`
- `POST /broadhub-adapter/v1/serial/rebind`
- `POST /broadhub-adapter/v1/serial/unbind`

Implementation rules:

- Resolve actor user and role from auth context, not from public request body.
- Check idempotency before inserting a new operation or creating a side effect.
- Return the existing operation/result for the same idempotency key and same request hash.
- Return `409 IDEMPOTENCY_KEY_REUSED` for the same key with a different request hash.
- Use transactions or unique constraints for concurrency-sensitive creation.

6. Data Loading

Load data in this order:

1. Authoritative aggregate state from the owning module.
2. Operator/module config from this DD's config tables.
3. Catalog/reference data from approved APIs or client wrappers.
4. Current in-flight operation state from the framework, where relevant.

Do not read current mutable state from cache for state-changing operations.

7. Drools Configuration

Packages found:

- No Drools package explicitly declared.

For each package:

- Define fact classes that mirror the request, current state, config, and reference data.
- Keep rule output as explicit decision fields.
- Unit-test accept and reject cases from the DD.
- BPMN should re-check critical rules after start if state may have changed.

8. BPMN And Worker Topics

Worker topics found:

- `bil01-charge-pre-authorize`
- `contractor-slot-reserve`
- `nrb-cmts-01`
- `payment-confirmed`

Implementation rules:

- Model service tasks with `camunda:type="external"` and the exact topic string.

- Worker handler must validate required variables before calling downstream services.
- Store outputs as process variables with stable names.
- Use message correlation keys exactly as specified by the DD.
- Add compensation paths for each external side effect.

9. Events

Events found:

- No event explicitly declared.

Event rules:

- Emit success only after authoritative commit.
- Emit rejected/failure event only after compensation/final failure decision.
- Use the outbox table/dispatcher; do not publish directly inside the transaction boundary unless the platform standard allows it.

10. Smoke Tests

Add tests for:

- Happy path.
- Invalid role or trigger.
- Invalid state.
- Missing config/catalog row.
- Dependency failure.
- Retry/idempotency.
- Event payload correctness.
- Worker topic failure and compensation.

11. Common Mistakes

- Treating worker-topic names as shell commands.
- Querying another module's database directly.
- Reading current state from cache in a state-changing flow.
- Emitting success before final commit.
- Hiding manual recovery paths for partial external side effects.
- Creating one deployment per topic when a consolidated worker app would be easier to operate.